Folds: Recursion as a Function

CS 350

Dr. Joseph Eremondi

February 21, 2025

Outline

Abstracting Generative Recursion

Abstracting Generative Recursion

Objective

- Goals

Objective

- Goals
  - To see how the patterns of normal and generative recursion on lists can be represented concretely as a higher order function

Objective

- Goals
  - To see how the patterns of normal and generative recursion on lists can be represented concretely as a higher order function
- Key Concepts

Objective

- Goals
  - To see how the patterns of normal and generative recursion on lists can be represented concretely as a higher order function
- Key Concepts
  - `foldl` and `foldr`

Higher-Order Genrative Recursion: Folds

- Recall: higher-order polymorphic functions let us turn design patterns into functions

Higher-Order Genrative Recursion: Folds

- Recall: higher-order polymorphic functions let us turn design patterns into functions
  - e.g. `map` for “do this to each element of a list”

Higher-Order Genrative Recursion: Folds

- Recall: higher-order polymorphic functions let us turn design patterns into functions
  - e.g. `map` for “do this to each element of a list”
  - `filter` for “Get rid of elements that fail some check”

Higher-Order Genrative Recursion: Folds

- Recall: higher-order polymorphic functions let us turn design patterns into functions
  - e.g. `map` for “do this to each element of a list”
  - `filter` for “Get rid of elements that fail some check”
- We had a recipe/template for tail recursion

Higher-Order Genrative Recursion: Folds

- Recall: higher-order polymorphic functions let us turn design patterns into functions
  - e.g. `map` for “do this to each element of a list”
  - `filter` for “Get rid of elements that fail some check”
- We had a recipe/template for tail recursion
  - We can turn this into a function

Recall: Tail-Recursion Template

- Make a helper with an extra argument

Recall: Tail-Recursion Template

- Make a helper with an extra argument
  - Extra arg called the *accumulator*

Recall: Tail-Recursion Template

- Make a helper with an extra argument
  - Extra arg called the *accumulator*
- Pattern match on the input you're processing

Recall: Tail-Recursion Template

- Make a helper with an extra argument
  - Extra arg called the *accumulator*
- Pattern match on the input you're processing
- Base case: return the accumulator

Recall: Tail-Recursion Template

- Make a helper with an extra argument
  - Extra arg called the *accumulator*
- Pattern match on the input you're processing
- Base case: return the accumulator
- Cons case: just call ourselves tail-recursively

Recall: Tail-Recursion Template

- Make a helper with an extra argument
  - Extra arg called the *accumulator*
- Pattern match on the input you're processing
- Base case: return the accumulator
- Cons case: just call ourselves tail-recursively
  - With one less element in the input list

Recall: Tail-Recursion Template

- Make a helper with an extra argument
  - Extra arg called the *accumulator*
- Pattern match on the input you're processing
- Base case: return the accumulator
- Cons case: just call ourselves tail-recursively
  - With one less element in the input list
  - With the updated accumulator

Foldl: The Idea

- We can make “update the accumulator” concrete as a function

Foldl: The Idea

- We can make “update the accumulator” concrete as a function
- Compute a new value for the accumulator based on:

Foldl: The Idea

- We can make “update the accumulator” concrete as a function
- Compute a new value for the accumulator based on:
  - Old value of the accumulator

Foldl: The Idea

- We can make “update the accumulator” concrete as a function
- Compute a new value for the accumulator based on:
  - Old value of the accumulator
  - Current element of the list

Foldl: The Idea

- We can make “update the accumulator” concrete as a function
- Compute a new value for the accumulator based on:
  - Old value of the accumulator
  - Current element of the list
- Take that function as an argument, then just use our template

Foldl: The Idea

- We can make “update the accumulator” concrete as a function
- Compute a new value for the accumulator based on:
  - Old value of the accumulator
  - Current element of the list
- Take that function as an argument, then just use our template
  - All that’s left is to give an initial value for the accumulator

Foldl (Fold Left)

Foldl (Fold Left)

```
(define (foldl [f : ('a 'b -> 'b)]
               [accum : 'b]
               [xs : (Listof 'a)]) : 'b
  (type-case (Listof 'a) xs
    [empty
      accum ]
    [(cons h t)
       (foldl f (f h accum) t)]))
```

- Polymorphic over:

Foldl (Fold Left)

```
(define (foldl [f : ('a 'b -> 'b)]
               [accum : 'b]
               [xs : (Listof 'a)]) : 'b
  (type-case (Listof 'a) xs
    [empty
      accum ]
    [(cons h t)
       (foldl f (f h accum) t)]))
```

- Polymorphic over:
  - List element type a

Foldl (Fold Left)

```
(define (foldl [f : ('a 'b -> 'b)]
               [accum : 'b]
               [xs : (Listof 'a)]) : 'b
  (type-case (Listof 'a) xs
    [empty
      accum ]
    [(cons h t)
       (foldl f (f h accum) t)]))
```

- Polymorphic over:
  - List element type a
  - Result type b

Foldl (Fold Left)

```
(define (foldl [f : ('a 'b -> 'b)]
               [accum : 'b]
               [xs : (Listof 'a)]) : 'b
  (type-case (Listof 'a) xs
    [empty
      accum ]
    [(cons h t)
       (foldl f (f h accum) t)]))
```

- Polymorphic over:
  - List element type a
  - Result type b
- If list empty, gives whatever the current `accum` is

Foldl (Fold Left)

```
(define (foldl [f : ('a 'b -> 'b)]
               [accum : 'b]
               [xs : (Listof 'a)]) : 'b
  (type-case (Listof 'a) xs
    [empty
      accum ]
    [(cons h t)
       (foldl f (f h accum) t)]))
```

- Polymorphic over:
  - List element type a
  - Result type b
- If list empty, gives whatever the current `accum` is
- If not empty, uses `f` to update the `accum` based on the first element of the list

Foldl (Fold Left)

```
(define (foldl [f : ('a 'b -> 'b)]
               [accum : 'b]
               [xs : (Listof 'a)]) : 'b
  (type-case (Listof 'a) xs
    [empty
      accum ]
    [(cons h t)
       (foldl f (f h accum) t)]))
```

- Polymorphic over:
  - List element type `a`
  - Result type `b`
- If list empty, gives whatever the current `accum` is
- If not empty, uses `f` to update the `accum` based on the first element of the list
  - Then processes the rest of the list with a recursive call

Foldl (Fold Left)

```
(define (foldl [f : ('a 'b -> 'b)]
               [accum : 'b]
               [xs : (Listof 'a)]) : 'b
  (type-case (Listof 'a) xs
    [empty
      accum ]
    [(cons h t)
       (foldl f (f h accum) t)]))
```

- Polymorphic over:
  - List element type `a`
  - Result type `b`
- If list empty, gives whatever the current `accum` is
- If not empty, uses `f` to update the `accum` based on the first element of the list
  - Then processes the rest of the list with a recursive call
- Tail recursive, so fast and takes constant space

Example: Reverse using folds

Example: Reverse using folds

```
(define (fold-reverse [xs : (Listof 'elType)]) : (Listof 'elType)
  (foldl (lambda (elem accum) (cons elem accum))
         empty
         xs))
(fold-reverse '(1 2 3))
```

Example: Reverse using folds

```
(define (fold-reverse [xs : (Listof 'elType)]) : (Listof 'elType)
  (foldl (lambda (elem accum) (cons elem accum))
         empty
         xs))
(fold-reverse '(1 2 3))
```

```
'(3 2 1)
```

- 'a is 'elType, 'b is (Listof 'elType)

Example: Reverse using folds

```
(define (fold-reverse [xs : (Listof 'elType)]) : (Listof 'elType)
  (foldl (lambda (elem accum) (cons elem accum))
         empty
         xs))
(fold-reverse '(1 2 3))
```

```
'(3 2 1)
```

- 'a is 'elType, 'b is (Listof 'elType)
- Update function type:

Example: Reverse using folds

```
(define (fold-reverse [xs : (Listof 'elType)]) : (Listof 'elType)
  (foldl (lambda (elem accum) (cons elem accum))
         empty
         xs))
(fold-reverse '(1 2 3))
```

```
'(3 2 1)
```

- 'a is 'elType, 'b is (Listof 'elType)
- Update function type:
  - ('elType (Listof 'elType) -> (Listof 'elType))

Example: Reverse using folds

```
(define (fold-reverse [xs : (Listof 'elType)]) : (Listof 'elType)
  (foldl (lambda (elem accum) (cons elem accum))
         empty
         xs))
(fold-reverse '(1 2 3))
```

```
'(3 2 1)
```

- `'a` is `'elType`, `'b` is `(Listof 'elType)`
- Update function type:
  - `('elType (Listof 'elType) -> (Listof 'elType))`
- Initial accumulator is empty list, since we start with nothing

Example: Reverse using folds

```
(define (fold-reverse [xs : (Listof 'elType)]) : (Listof 'elType)
  (foldl (lambda (elem accum) (cons elem accum))
         empty
         xs))
(fold-reverse '(1 2 3))
```

```
'(3 2 1)
```

- `'a` is `'elType`, `'b` is `(Listof 'elType)`
- Update function type:
  - `('elType (Listof 'elType) -> (Listof 'elType))`
- Initial accumulator is empty list, since we start with nothing
- Update function adds the current element to the front of the list we've built so far

Reasoning with foldl

- `(foldl f init (list a b c d))` gives:

Reasoning with foldl

- `(foldl f init (list a b c d))` gives:
  - `(f d (f c (f b (f a init))))`

Reasoning with foldl

- `(foldl f init (list a b c d))` gives:
  - `(f d (f c (f b (f a init))))`
- Similar to normal recursion:

Reasoning with foldl

- `(foldl f init (list a b c d))` gives:
  - `(f d (f c (f b (f a init))))`
- Similar to normal recursion:
  - Assume that `accum` has the correct result so far

Reasoning with foldl

- `(foldl f init (list a b c d))` gives:
  - `(f d (f c (f b (f a init))))`
- Similar to normal recursion:
  - Assume that `accum` has the correct result so far
  - Find the `f` that incorporates the next element of the list into the new `accum`

Reasoning with foldl

- `(foldl f init (list a b c d))` gives:
  - `(f d (f c (f b (f a init))))`
- Similar to normal recursion:
  - Assume that `accum` has the correct result so far
  - Find the `f` that incorporates the next element of the list into the new `accum`
- For `fold-reverse`

Reasoning with foldl

- `(foldl f init (list a b c d))` gives:
  - `(f d (f c (f b (f a init))))`
- Similar to normal recursion:
  - Assume that `accum` has the correct result so far
  - Find the `f` that incorporates the next element of the list into the new `accum`
- For `fold-reverse`
  - Assume that `accum` has the correct reversal of the list we've seen so far. How do we incorporate the next element?

Reasoning with foldl

- `(foldl f init (list a b c d))` gives:
  - `(f d (f c (f b (f a init))))`
- Similar to normal recursion:
  - Assume that `accum` has the correct result so far
  - Find the `f` that incorporates the next element of the list into the new `accum`
- For `fold-reverse`
  - Assume that `accum` has the correct reversal of the list we've seen so far. How do we incorporate the next element?
    - We add it to the front, since it's the last of all elements we've seen so far, so the first in the reversed list

Reasoning with foldl

- `(foldl f init (list a b c d))` gives:
  - `(f d (f c (f b (f a init))))`
- Similar to normal recursion:
  - Assume that `accum` has the correct result so far
  - Find the `f` that incorporates the next element of the list into the new `accum`
- For `fold-reverse`
  - Assume that `accum` has the correct reversal of the list we've seen so far. How do we incorporate the next element?
    - We add it to the front, since it's the last of all elements we've seen so far, so the first in the reversed list
    - Recursive call says "keep processing for the rest of the list"

Foldl is a for-loop

Foldl is a for-loop

```
int A[N];
int x;
// and some other variables
for (int i = 0; i < N; i++){
  int current = A[i]
  // Do something with x
 }
```

- C++ for-loop over array

Foldl is a for-loop

```
int A[N];
int x;
// and some other variables
for (int i = 0; i < N; i++){
  int current = A[i]
  // Do something with x
 }
```

- C++ for-loop over array
  - Get the array element for each `i` in `0` ... N

Foldl is a for-loop

```
int A[N];
int x;
// and some other variables
for (int i = 0; i < N; i++){
  int current = A[i]
  // Do something with x
 }
```

- C++ for-loop over array
  - Get the array element for each `i` in `0` ... N
  - Do some stuff

Foldl is a for-loop

```
int A[N];
int x;
// and some other variables
for (int i = 0; i < N; i++){
  int current = A[i]
  // Do something with x
 }
```

- C++ for-loop over array
  - Get the array element for each `i` in `0` … N
  - Do some stuff
    - e.g. Set the values of the other variables

Foldl is a for-loop

```
int A[N];
int x;
// and some other variables
for (int i = 0; i < N; i++){
  int current = A[i]
  // Do something with x
 }
```

- C++ for-loop over array
  - Get the array element for each `i` in `0` … `N`
  - Do some stuff
    - e.g. Set the values of the other variables
- Flit fold is this but with *explicit state*

Foldl is a for-loop

```
int A[N];
int x;
// and some other variables
for (int i = 0; i < N; i++){
  int current = A[i]
  // Do something with x
 }
```

- C++ for-loop over array
  - Get the array element for each `i` in `0` … `N`
  - Do some stuff
    - e.g. Set the values of the other variables
- Flit fold is this but with *explicit state*
  - Accumulator captures all the variables that the loop can modify

Foldl is a for-loop

```
int A[N];
int x;
// and some other variables
for (int i = 0; i < N; i++){
  int current = A[i]
  // Do something with x
 }
```

- C++ for-loop over array
  - Get the array element for each `i` in `0` … `N`
  - Do some stuff
    - e.g. Set the values of the other variables
- Flit fold is this but with *explicit state*
  - Accumulator captures all the variables that the loop can modify
  - New variable value come from function

Foldl is a for-loop

```
int A[N];
int x;
// and some other variables
for (int i = 0; i < N; i++){
  int current = A[i]
  // Do something with x
 }
```

- C++ for-loop over array
  - Get the array element for each `i` in `0` … `N`
  - Do some stuff
    - e.g. Set the values of the other variables
- Flit fold is this but with *explicit state*
  - Accumulator captures all the variables that the loop can modify
  - New variable value come from function
  - Multiple variables? Use pair types

Example: Fibonacci with Foldl

- See Dr. Racket

Non-tail Folds

- Recall our pattern for normal (non-tail) recursion

Non-tail Folds

- Recall our pattern for normal (non-tail) recursion
- Pattern match on the input:

Non-tail Folds

- Recall our pattern for normal (non-tail) recursion
- Pattern match on the input:
    - Base case: return simplest result

Non-tail Folds

- Recall our pattern for normal (non-tail) recursion
- Pattern match on the input:
  - Base case: return simplest result
  - Recursive case: call self on tail, then put together with head

Non-tail Folds

- Recall our pattern for normal (non-tail) recursion
- Pattern match on the input:
  - Base case: return simplest result
  - Recursive case: call self on tail, then put together with head
- Can make a higher-order function to generalize this

Non-tail Folds

- Recall our pattern for normal (non-tail) recursion
- Pattern match on the input:
  - Base case: return simplest result
  - Recursive case: call self on tail, then put together with head
- Can make a higher-order function to generalize this
  - Function parameter for the “put together” part

Constructing Foldr (Fold Right)

Constructing Foldr (Fold Right)

```
(define (foldr [f : ('a 'b -> 'b)]
               [init : 'b]
               [xs : (Listof 'a)]) : 'b
  (type-case (Listof 'a) xs
    [empty
      init ]
    [(cons h t)
       (f h (foldr f init t))]))
```

- Like foldr, but processes the list right-to-left

Constructing Foldr (Fold Right)

```
(define (foldr [f : ('a 'b -> 'b)]
               [init : 'b]
               [xs : (Listof 'a)]) : 'b
  (type-case (Listof 'a) xs
    [empty
      init ]
    [(cons h t)
       (f h (foldr f init t))]))
```

- Like foldr, but processes the list right-to-left
- `(foldr f init '(a b c d))` gives `(f a (f b (f c (f d init))))`

Constructing Foldr (Fold Right)

```
(define (foldr [f : ('a 'b -> 'b)]
               [init : 'b]
               [xs : (Listof 'a)]) : 'b
  (type-case (Listof 'a) xs
    [empty
      init ]
    [(cons h t)
       (f h (foldr f init t))]))
```

- Like foldr, but processes the list right-to-left
- `(foldr f init '(a b c d))` gives `(f a (f b (f c (f d init))))`
- Reasoning the exact same, except we assume the argument to `f` has the accumulator for all elements *after* the current element

Constructing Foldr (Fold Right)

```
(define (foldr [f : ('a 'b -> 'b)]
               [init : 'b]
               [xs : (Listof 'a)]) : 'b
  (type-case (Listof 'a) xs
    [empty
      init ]
    [(cons h t)
       (f h (foldr f init t))]))
```

- Like foldr, but processes the list right-to-left
- `(foldr f init '(a b c d))` gives `(f a (f b (f c (f d init))))`
- Reasoning the exact same, except we assume the argument to `f` has the accumulator for all elements *after* the current element
- Generally `foldl` is faster

Example: Map as a fold

Example: Map as a fold

```
(define (map-via-fold [f : ('a -> 'b)]
                      [xs : (Listof 'a)])
  : (Listof 'b)
  (foldr (lambda (elem mappedTail)
           (cons (f elem) mappedTail))
         '()
         xs))
(map-via-fold add1 (list 2 3 4 5))
```

Example: Map as a fold

```
(define (map-via-fold [f : ('a -> 'b)]
                      [xs : (Listof 'a)])
  : (Listof 'b)
  (foldr (lambda (elem mappedTail)
           (cons (f elem) mappedTail))
         '()
         xs))
(map-via-fold add1 (list 2 3 4 5))
```

```
'(3 4 5 6)
```

List Recursion, Once And For All

- `foldr` is the higher-order function version of pattern matching on lists

List Recursion, Once And For All

- `foldr` is the higher-order function version of pattern matching on lists
  - Give an empty case (initial value)

List Recursion, Once And For All

- `foldr` is the higher-order function version of pattern matching on lists
  - Give an empty case (initial value)
  - Give a function that incorporates head of the list with the recursive result on the rest of the list

List Recursion, Once And For All

- `foldr` is the higher-order function version of pattern matching on lists
  - Give an empty case (initial value)
  - Give a function that incorporates head of the list with the recursive result on the rest of the list
- `foldl` is the higher-order function version of generative recursion on lists

List Recursion, Once And For All

- `foldr` is the higher-order function version of pattern matching on lists
  - Give an empty case (initial value)
  - Give a function that incorporates head of the list with the recursive result on the rest of the list
- `foldl` is the higher-order function version of generative recursion on lists
  - Give an initial accumulator

List Recursion, Once And For All

- `foldr` is the higher-order function version of pattern matching on lists
  - Give an empty case (initial value)
  - Give a function that incorporates head of the list with the recursive result on the rest of the list
- `foldl` is the higher-order function version of generative recursion on lists
  - Give an initial accumulator
  - Give a way to update the accumulator for each element of the list

List Recursion, Once And For All

- `foldr` is the higher-order function version of pattern matching on lists
  - Give an empty case (initial value)
  - Give a function that incorporates head of the list with the recursive result on the rest of the list
- `foldl` is the higher-order function version of generative recursion on lists
  - Give an initial accumulator
  - Give a way to update the accumulator for each element of the list
- These two functions can replace (nearly) all pattern matching/recursion on lists

List Recursion, Once And For All

- `foldr` is the higher-order function version of pattern matching on lists
  - Give an empty case (initial value)
  - Give a function that incorporates head of the list with the recursive result on the rest of the list
- `foldl` is the higher-order function version of generative recursion on lists
  - Give an initial accumulator
  - Give a way to update the accumulator for each element of the list
- These two functions can replace (nearly) all pattern matching/recursion on lists
- Examples (see lecture Racket file)

Folds for other types

- `foldl` only works with lists or list-like types

Folds for other types

- `foldl` only works with lists or list-like types
  - If there's multiple recursive values (e.g. tree), then can't do only tail calls

Folds for other types

- `foldl` only works with lists or list-like types
  - If there's multiple recursive values (e.g. tree), then can't do only tail calls
- Conversely, every datatype has a version of `foldr`

Folds for other types

- `foldl` only works with lists or list-like types
  - If there's multiple recursive values (e.g. tree), then can't do only tail calls
- Conversely, every datatype has a version of `foldr`
  - Take a function that tells how to put recursive results into a final result

Folds for other types

- `foldl` only works with lists or list-like types
  - If there's multiple recursive values (e.g. tree), then can't do only tail calls
- Conversely, every datatype has a version of `foldr`
  - Take a function that tells how to put recursive results into a final result
  - Could derive it automatically, but Flit doesn't do this.

Folds for other types

- `foldl` only works with lists or list-like types
  - If there's multiple recursive values (e.g. tree), then can't do only tail calls
- Conversely, every datatype has a version of `foldr`
  - Take a function that tells how to put recursive results into a final result
  - Could derive it automatically, but Flit doesn't do this.
  - Pattern matching is usually easier for non-lists